

FinanceFlow

Análise Técnica Detalhada

Gerado em 10/08/2026

Funcionalidades, telas e todas as fórmulas de cálculo usadas no app.

0. Sumário

- 1. Visão geral e navegação
- 2. Dashboard Principal (Início)
- 3. Dívidas
- 4. Relatórios
- 5. Registrar (ganhos e gastos, leitura de nota fiscal por IA)
- 6. Configurações
- 7. Glossário de fórmulas
- 8. Rotas de API e automações

1. Visão Geral e Navegação

FinanceFlow é um app de controle financeiro pessoal construído em Next.js 14 (App Router) com Tailwind CSS e Supabase (banco de dados Postgres, autenticação e storage). Todo o cálculo financeiro roda em funções puras centralizadas em `lib/financas.ts`, usadas tanto pelas páginas quanto pelas rotas de API — garantindo que o mesmo número seja calculado da mesma forma em qualquer lugar do app.

A navegação principal (menu inferior) tem 4 itens:

- Início — dashboard simplificado do dia (ganho, gasto, sobrou, projeção do mês).
- Dívidas — dashboard dedicado para gerenciar dívidas (separar dinheiro, marcar como paga).
- Relatórios — resumo do período, análise financeira, gráficos, busca de gastos, exportação em PDF.
- Configurações — meta diária e orçamentos por categoria (usados nos cálculos de alerta).

2. Dashboard Principal (Início)

A tela inicial mostra apenas dados do dia atual, sem distrações — dívidas, alertas e metas ficam em outras telas.

Ganho do dia / Gastos do dia

Soma simples dos valores de ganhos (ou gastos) cuja data é igual à data de hoje.

```
Ganho_Dia = soma(valor) de todos os ganhos onde data = hoje  
Gasto_Dia = soma(valor) de todos os gastos onde data = hoje
```

Sobrou no dia

```
Sobrou_Dia = Ganho_Dia - Gasto_Dia
```

Exibido em verde quando ≥ 0 , em vermelho quando negativo.

Projeção do mês

Estima quanto o usuário terá ganho até o fim do mês, assumindo que o ritmo de ganho dos dias já passados se mantém constante nos dias restantes.

```
Media_Diaria = soma(ganhos do mes, do dia 1 ate hoje) / dia_atual_do_mes  
Projecao = Media_Diaria * dias_totais_no_mes  
"Se continuar assim, tera R$ Projecao ate DD/MM"
```

`dias_totais_no_mes` é calculado pegando o dia 0 do mês seguinte (truque de calendário que sempre resulta no último dia do mês atual, incluindo meses de 28, 29, 30 ou 31 dias).

3. Dívidas

Cada dívida tem: `credor`, `valor_total`, `valor_pago` (quanto já foi efetivamente pago), `valor_separado` (quanto o usuário guardou mas ainda não pagou), `tipo` (única ou recorrente), `status` (aberta ou paga) e, para recorrentes, um `dia_recorrencia` (1-31) e `prazo_vencimento`.

Falta Pagar (por dívida)

```
Falta_Pagar = max(0, valor_total - valor_pago)
```

Importante: `valor_separado` NÃO é descontado aqui. Dinheiro separado só sai do total quando a dívida é de fato marcada como paga — até lá ele aparece só como referência de quanto já foi guardado.

Resumo do topo da página (soma de todas as dívidas com status = aberta)

```
Total_Dividias = soma(valor_total) das dividas abertas
Ja_Separado     = soma(valor_separado) das dividas abertas
Falta_Pagar     = soma(max(0, valor_total - valor_pago)) das dividas abertas
```

Botão "+ Separar para Dívida"

Chama POST `/api/dividas/:id/separar` com o valor informado. O valor é somado ao `valor_separado` atual da dívida e um registro histórico é gravado na tabela `separacoes_divida`. Não afeta `valor_pago` nem o status da dívida.

```
novo_valor_separado = valor_separado_atual + valor_informado
```

Botão "Marcar como Paga"

Chama PATCH `/api/dividas/:id/pagar` enviando como `valor_pago` exatamente o `Falta_Pagar` da dívida (ou seja, sempre quita o saldo inteiro de uma vez). O comportamento depois de quitar depende do tipo:

- Única: status vira 'paga', `valor_separado` zera, a dívida sai da lista ativa e passa para a seção 'Dívidas Pagas'.
- Recorrente: `valor_pago` volta a 0, `valor_separado` zera, status continua 'aberta' e `prazo_vencimento` avança para a próxima ocorrência do `dia_recorrencia` (ver fórmula de recorrência abaixo) — a dívida continua na lista, pronta para o próximo ciclo.

Um pagamento parcial (valor menor que o total) também é aceito pela API, mas a interface atual sempre envia o valor cheio ao clicar em 'Marcar como Paga'.

Cálculo da próxima data de recorrência

Usado tanto ao criar uma dívida recorrente (primeiro vencimento) quanto ao renová-la (próximo ciclo). Se o `dia_recorrencia` não existe no mês (ex: dia 31 em fevereiro), é ajustado ("clampado") para o último dia do mês.

```
candidato_este_mes = clamp(dia_recorrencia, ultimo_dia_do_mes_atual)

se estritamenteApos = false (criando a divida):
  usa candidato_este_mes se ele for >= data de referencia
  senao avanca para o mesmo dia (clampado) do mes seguinte

se estritamenteApos = true (renovando um ciclo):
  usa candidato_este_mes so se ele for ESTRITAMENTE MAIOR que a referencia
  senao avanca sempre para o mes seguinte
```

Status visual (recorrente): Em dia vs Vencida

```
status = (prazo_vencimento >= hoje) ? "Em dia" (verde) : "Vencida" (vermelho)
```

Renovação automática por cron

Além da renovação ao pagar, existe uma rotina diária (GET /api/cron/renovar-dividas, protegida por CRON_SECRET, agendada às 06:00 UTC via vercel.json) que varre todas as dívidas recorrentes com prazo_vencimento já vencido e as renova mesmo sem pagamento: valor_pago volta a 0 e prazo_vencimento avança para a próxima ocorrência do dia_recorrencia.

4. Relatórios

A página tem um seletor Semana/Mês que define o período usado no Resumo, nos Alertas, em Gastos por Categoria e em Gastos Idiotas. Os Gráficos e a Análise Financeira usam sempre o mês corrente inteiro, independente do seletor.

Resumo do período

```
Ganhos = soma(valor) dos ganhos no periodo selecionado
Gastos = soma(valor) dos gastos no periodo selecionado
Saldo = Ganhos - Gastos
```

Análise Financeira (comparativo com o mês anterior)

Sempre compara o mês corrente completo com o mês anterior completo (não depende do seletor Semana/Mês).

```
variacao_ganhos = (Ganhos_MesAtual - Ganhos_MesAnterior) / Ganhos_MesAnterior * 100
variacao_gastos = (Gastos_MesAtual - Gastos_MesAnterior) / Gastos_MesAnterior * 100
(se o mes anterior nao teve ganhos/gastos, a variacao correspondente fica indefinida)
```

Exibido como texto: "Você ganhou X% a mais/a menos que o mês passado" (mesma lógica para gastos, mas verde quando gastou menos, vermelho quando gastou mais).

Score de Eficiência (0 a 10)

Mede a taxa de economia do mês corrente: quanto do que foi ganho sobrou sem ser gasto.

```
taxa_economia = clamp((Ganhos_MesAtual - Gastos_MesAtual) / Ganhos_MesAtual, 0, 1)
Score = arredondar(taxa_economia * 10)

Se Ganhos_MesAtual = 0, Score = 0.
```

Faixa	Rótulo
8 a 10	Excelente
5 a 7	Bom
3 a 4	Regular
0 a 2	Atenção

Alertas de orçamento ultrapassado

Compara o gasto por categoria no período com os limites definidos em Configurações (Alimentação, Higiene, Contas).

```
se Gasto_Categoria > Orcamento_Categoria: alerta com Ultrapassou = Gasto_Categoria - Orcamento_Categoria
```

Gastos por categoria

Soma o valor de todos os gastos do período, agrupados por categoria, ordenado do maior para o menor.

"Gastos Idiotas"

```
Gasto_Idiota = gasto onde eh_desnecessario = true OU categoria = "Gasto Idiota"
```

Gráficos (sempre referentes ao mês corrente)

- Pizza — gastos por categoria do mês.
- Barra — ganhos vs. gastos, um par de barras por dia do mês.
- Linha — saldo acumulado dia a dia: para cada dia do mês, soma (ganhos do dia - gastos do dia) ao acumulado do dia anterior, formando a curva de evolução do saldo ao longo do mês.

Buscar Gastos

Filtros combináveis (busca por texto na descrição, categoria, intervalo de datas, valor mínimo/máximo) enviados para GET /api/gastos/filtrado, que monta a consulta dinamicamente e devolve os gastos junto com a soma total encontrada.

Ganhos por Período

Duas formas de consulta, via GET /api/ganhos/filtrado:

- Por dia da semana — soma todos os ganhos históricos que caíram numa segunda, terça etc., e calcula a média por número de dias distintos com ganho naquele dia da semana.
- Por intervalo de datas — soma os ganhos no intervalo e divide pelo número de dias do intervalo (média diária).

Exportar PDF

Gera um PDF no navegador (jsPDF) com resumo do período, tabela de gastos por categoria (texto) e uma captura de tela dos três gráficos (html2canvas), tudo processado localmente sem passar pelo servidor.

5. Registrar (Ganhos e Gastos)

Formulário simples para lançar um ganho ou gasto: valor, descrição, data e, no caso de gasto, categoria e a opção de marcar como gasto desnecessário.

Leitura de nota fiscal por IA (Claude Vision)

Ao enviar uma foto de nota fiscal/recibo, a imagem é enviada para a API da Anthropic (modelo claude-opus-5) com uma saída estruturada (JSON Schema) que força a IA a devolver uma lista de itens, cada um com descrição, valor e categoria (escolhida entre as categorias válidas do app). Se não for possível separar por item, a IA devolve um único item com o valor total da nota e o nome do estabelecimento. O usuário revisa e confirma antes de qualquer gasto ser realmente salvo.

6. Configurações

Tela onde o usuário define os parâmetros usados pelos cálculos acima:

Campo	Onde é usado
meta_diaria	Comparação de referência de ganho diário (uso histórico do app)
orcamento_alimentacao	Alerta de orçamento ultrapassado na categoria Alimentação
orcamento_higiene	Alerta de orçamento ultrapassado na categoria Higiene
orcamento_contas	Alerta de orçamento (Contas) e no cálculo Falta_para_Contas

Ao criar a conta, um trigger no banco cria automaticamente uma linha de configurações padrão para o novo usuário.

7. Glossário de Fórmulas

```
Ganho_Dia          = soma(ganhos de hoje)
Gasto_Dia          = soma(gastos de hoje)
Sobrou_Dia         = Ganho_Dia - Gasto_Dia

Media_Diaria       = soma(ganhos do mes ate hoje) / dia_atual_do_mes
Projecao_Mes       = Media_Diaria * dias_totais_no_mes

Falta_Pagar(divida)= max(0, valor_total - valor_pago)
Total_Dividas      = soma(valor_total) das dividas abertas
Ja_Separado        = soma(valor_separado) das dividas abertas

Variacao_%         = (Atual - Anterior) / Anterior * 100
Taxa_Economia      = clamp((Ganhos - Gastos) / Ganhos, 0, 1)
Score_Eficiencia   = arredondar(Taxa_Economia * 10) // 0 a 10

Saldo_Acumulado(d) = Saldo_Acumulado(d-1) + Ganho(d) - Gasto(d)

Falta_para_Contas  = max(0, orcamento_contas - (Ganhos_Mes - Gastos_Mes))
```

Observação sobre Falta_para_Contas: essa fórmula ainda existe em `lib/financas.ts` e é usada pela rota `GET /api/dashboard`, mas não é mais exibida na tela inicial — o dashboard principal foi simplificado para mostrar só o resumo do dia.

8. Rotas de API e Automações

Rota	Função
GET /api/dashboard	Resumo do dia (ganho, gasto, sobrou)
GET /api/ganhos/filtrado	Ganhos por dia da semana ou por intervalo de datas
GET /api/gastos/filtrado	Busca de gastos com filtros combinados
POST /api/dividas/:id/separar	Soma valor ao valor_separado da dívida
PATCH /api/dividas/:id/pagar	Registra pagamento; quita e renova/arquiva a dívida
GET /api/cron/renovar-dividas	Job diário que renova recorrentes vencidas (CRON_SECRET)
POST /api/nota-fiscal	Leitura de nota fiscal via Claude Vision

Todas as rotas (exceto o cron, protegido por CRON_SECRET) exigem sessão autenticada do Supabase e aplicam Row Level Security: cada usuário só enxerga e altera os próprios registros (auth.uid() = usuario_id).